

```

# =====
# 7. FEATURE MATCHING USING SIFT
# Feature Extraction and Matching using SIFT + FLANN
# =====

import cv2
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import time
from google.colab import files

# =====
# STEP 1: UPLOAD TWO IMAGES
# =====

print("Upload Image 1 (Reference Image)")
# files.upload() # Removed as files are already uploaded

print("Upload Image 2 (Test Image)")
# files.upload() # Removed as files are already uploaded

# Change these names if your uploaded files have different names
image1_path = "REF 1.png"
image2_path = "REF 2.png"

# =====
# STEP 2: READ THE IMAGES
# =====

img1 = cv2.imread(image1_path)
img2 = cv2.imread(image2_path)

if img1 is None or img2 is None:
    raise FileNotFoundError(
        "Check that REF 1.png and REF 2.png exist."
    )

# Resize images
img1 = cv2.resize(img1, (800, 600))
img2 = cv2.resize(img2, (800, 600))

# Convert to grayscale
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

# =====
# STEP 3: DISPLAY ORIGINAL IMAGES
# =====

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img1, cv2.COLOR_BGR2RGB))
plt.title("Image 1 - Reference")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(img2, cv2.COLOR_BGR2RGB))
plt.title("Image 2 - Test")
plt.axis("off")

```

```

plt.tight_layout()
plt.show()

# =====
# STEP 4: CREATE SIFT DETECTOR
# =====

sift = cv2.SIFT_create(
    nfeatures=1500
)

# =====
# STEP 5: DETECT SIFT KEYPOINTS AND DESCRIPTORS
# =====

start_time = time.perf_counter()

keypoints1, descriptors1 = sift.detectAndCompute(
    gray1,
    None
)

keypoints2, descriptors2 = sift.detectAndCompute(
    gray2,
    None
)

feature_time = time.perf_counter() - start_time

print("\n=====")
print("          SIFT FEATURE EXTRACTION")
print("=====")

print(
    "Keypoints in Image 1:",
    len(keypoints1)
)

print(
    "Keypoints in Image 2:",
    len(keypoints2)
)

print(
    "Feature extraction time:",
    round(feature_time * 1000, 2),
    "ms"
)

# =====
# STEP 6: DRAW SIFT KEYPOINTS
# =====

keypoint_image1 = cv2.drawKeypoints(
    img1,
    keypoints1,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

keypoint_image2 = cv2.drawKeypoints(
    img2,
    keypoints2,

```

```

None,
flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plt.imshow(
    cv2.cvtColor(
        keypoint_image1,
        cv2.COLOR_BGR2RGB
    )
)
plt.title(
    "SIFT Keypoints - Image 1"
)
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(
    cv2.cvtColor(
        keypoint_image2,
        cv2.COLOR_BGR2RGB
    )
)
plt.title(
    "SIFT Keypoints - Image 2"
)
plt.axis("off")

plt.tight_layout()
plt.show()

# =====
# STEP 7: FLANN FEATURE MATCHER
# =====

FLANN_INDEX_KDTREE = 1

index_params = {
    "algorithm": FLANN_INDEX_KDTREE,
    "trees": 5
}

search_params = {
    "checks": 50
}

flann = cv2.FlannBasedMatcher(
    index_params,
    search_params
)

# =====
# STEP 8: MATCH SIFT DESCRIPTORS
# =====

start_match = time.perf_counter()

matches = flann.knnMatch(
    descriptors1,
    descriptors2,
    k=2
)

```

```

match_time = time.perf_counter() - start_match

# =====
# STEP 9: LOWE'S RATIO TEST
# =====

good_matches = []

ratio_threshold = 0.75

for pair in matches:

    if len(pair) == 2:

        m, n = pair

        if m.distance < ratio_threshold * n.distance:
            good_matches.append(m)

# =====
# STEP 10: MATCHING STATISTICS
# =====

total_matches = len(matches)

good_match_count = len(good_matches)

match_percentage = (
    good_match_count / total_matches * 100
    if total_matches > 0
    else 0
)

print("\n=====")
print("          FEATURE MATCHING RESULTS")
print("=====")

print(
    "Total candidate matches:",
    total_matches
)

print(
    "Good matches:",
    good_match_count
)

print(
    "Good match percentage:",
    round(match_percentage, 2),
    "%"
)

print(
    "Matching time:",
    round(match_time * 1000, 2),
    "ms"
)

# =====
# STEP 11: DRAW GOOD MATCHES

```

```
# =====

matched_image = cv2.drawMatches(
    img1,
    keypoints1,
    img2,
    keypoints2,
    good_matches,
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(16, 8))

plt.imshow(
    cv2.cvtColor(
        matched_image,
        cv2.COLOR_BGR2RGB
    )
)

plt.title(
    "SIFT Feature Matching - Good Matches"
)

plt.axis("off")

plt.show()

# =====
# STEP 12: HOMOGRAPHY AND INLIER ANALYSIS
# =====
# Homography helps identify geometrically consistent matches.

inlier_count = 0
inlier_percentage = 0

if len(good_matches) >= 4:

    src_points = np.float32([
        keypoints1[m.queryIdx].pt
        for m in good_matches
    ]).reshape(-1, 1, 2)

    dst_points = np.float32([
        keypoints2[m.trainIdx].pt
        for m in good_matches
    ]).reshape(-1, 1, 2)

    H, mask = cv2.findHomography(
        src_points,
        dst_points,
        cv2.RANSAC,
        5.0
    )

    if mask is not None:

        inlier_count = int(
            mask.sum()
        )

        inlier_percentage = (
            inlier_count /
            len(good_matches)

```

```

        ) * 100

print(
    "Geometrically consistent matches:",
    inlier_count
)

print(
    "Inlier percentage:",
    round(
        inlier_percentage,
        2
    ),
    "%"
)

# =====
# STEP 13: FUNCTION FOR SIFT ROBUSTNESS TEST
# =====

def sift_match(reference, test, condition):

    start = time.perf_counter()

    kp1, des1 = sift.detectAndCompute(
        reference,
        None
    )

    kp2, des2 = sift.detectAndCompute(
        test,
        None
    )

    if des1 is None or des2 is None:

        return {
            "Condition": condition,
            "Keypoints 1": len(kp1),
            "Keypoints 2": len(kp2),
            "Good Matches": 0,
            "Match Percentage": 0,
            "Inliers": 0,
            "Inlier Percentage": 0,
            "Time (ms)": round(
                (time.perf_counter() - start) * 1000,
                2
            )
        }

    # FLANN matcher
    local_flann = cv2.FlannBasedMatcher(
        index_params,
        search_params
    )

    knn_matches = local_flann.knnMatch(
        des1,
        des2,
        k=2
    )

    good = []

```

```

for pair in knn_matches:

    if len(pair) == 2:

        m, n = pair

        if m.distance < 0.75 * n.distance:
            good.append(m)

# Homography
inliers = 0

if len(good) >= 4:

    src = np.float32([
        kp1[m.queryIdx].pt
        for m in good
    ]).reshape(-1, 1, 2)

    dst = np.float32([
        kp2[m.trainIdx].pt
        for m in good
    ]).reshape(-1, 1, 2)

    H, mask = cv2.findHomography(
        src,
        dst,
        cv2.RANSAC,
        5.0
    )

    if mask is not None:
        inliers = int(mask.sum())

good_percentage = (
    len(good) /
    len(knn_matches) * 100
    if len(knn_matches) > 0
    else 0
)

inlier_percentage = (
    inliers /
    len(good) * 100
    if len(good) > 0
    else 0
)

elapsed = (
    time.perf_counter() - start
) * 1000

return {
    "Condition": condition,
    "Keypoints 1": len(kp1),
    "Keypoints 2": len(kp2),
    "Good Matches": len(good),
    "Match Percentage": round(
        good_percentage,
        2
    ),
    "Inliers": inliers,
    "Inlier Percentage": round(
        inlier_percentage,
        2
    ),
    "Time (ms)": round(

```

```

        elapsed,
        2
    )
}

# =====
# STEP 14: CREATE SCALE VARIATION
# =====

scale_image = cv2.resize(
    gray1,
    None,
    fx=1.5,
    fy=1.5
)

scale_image = cv2.resize(
    scale_image,
    (800, 600)
)

# =====
# STEP 15: CREATE ROTATION VARIATIONS
# =====

height, width = gray1.shape

center = (
    width // 2,
    height // 2
)

rotation_30_matrix = cv2.getRotationMatrix2D(
    center,
    30,
    1.0
)

rotation_60_matrix = cv2.getRotationMatrix2D(
    center,
    60,
    1.0
)

rotation_30 = cv2.warpAffine(
    gray1,
    rotation_30_matrix,
    (width, height)
)

rotation_60 = cv2.warpAffine(
    gray1,
    rotation_60_matrix,
    (width, height)
)

# =====
# STEP 16: CREATE ILLUMINATION VARIATIONS
# =====

# Dark image
dark_image = cv2.convertScaleAbs(
    gray1,
    alpha=0.5

```



```
alpha=0.5,
beta=0
)

# Bright image
bright_image = cv2.convertScaleAbs(
    gray1,
    alpha=1.5,
    beta=50
)

# =====
# STEP 17: RUN ROBUSTNESS EXPERIMENT
# =====

robustness_results = []

robustness_results.append(
    sift_match(
        gray1,
        gray2,
        "Original"
    )
)

robustness_results.append(
    sift_match(
        gray1,
        scale_image,
        "Scale 1.5x"
    )
)

robustness_results.append(
    sift_match(
        gray1,
        rotation_30,
        "Rotation 30°"
    )
)

robustness_results.append(
    sift_match(
        gray1,
        rotation_60,
        "Rotation 60°"
    )
)

robustness_results.append(
    sift_match(
        gray1,
        dark_image,
        "Low Illumination"
    )
)

robustness_results.append(
    sift_match(
        gray1,
        bright_image,
        "High Illumination"
    )
)
```

```
# =====
# STEP 18: DISPLAY RESULTS TABLE
# =====

results_df = pd.DataFrame(
    robustness_results
)

print("\n=====")
print("          SIFT ROBUSTNESS EXPERIMENT")
print("=====")

print(
    results_df.to_string(
        index=False
    )
)

# =====
# STEP 19: GRAPH - GOOD MATCHES
# =====

plt.figure(figsize=(11, 6))

plt.bar(
    results_df["Condition"],
    results_df["Good Matches"]
)

plt.xlabel(
    "Image Condition"
)

plt.ylabel(
    "Number of Good Matches"
)

plt.title(
    "SIFT Good Matches under Different Conditions"
)

plt.xticks(
    rotation=25
)

plt.tight_layout()

plt.show()

# =====
# STEP 20: GRAPH - MATCH PERCENTAGE
# =====

plt.figure(figsize=(11, 6))

plt.plot(
    results_df["Condition"],
    results_df["Match Percentage"],
    marker="o",
    linewidth=2
)

plt.xlabel(
    "Image Condition"
)
```

```
plt.ylabel(
    "Good Match Percentage (%)"
)

plt.title(
    "SIFT Matching Performance"
)

plt.xticks(
    rotation=25
)

plt.grid(True)

plt.tight_layout()

plt.show()

# =====
# STEP 21: GRAPH - INLIER PERCENTAGE
# =====

plt.figure(figsize=(11, 6))

plt.bar(
    results_df["Condition"],
    results_df["Inlier Percentage"]
)

plt.xlabel(
    "Image Condition"
)

plt.ylabel(
    "Geometric Inlier Percentage (%)"
)

plt.title(
    "Geometrically Consistent SIFT Matches"
)

plt.xticks(
    rotation=25
)

plt.tight_layout()

plt.show()

# =====
# STEP 22: SAVE RESULTS
# =====

results_df.to_csv(
    "SIFT_matching_results.csv",
    index=False
)

cv2.imwrite(
    "SIFT_good_matches.jpg",
    matched_image
)
```

```
cv2.imwrite(
    "SIFT_keypoints_image1.jpg",
    keypoint_image1
)

cv2.imwrite(
    "SIFT_keypoints_image2.jpg",
    keypoint_image2
)

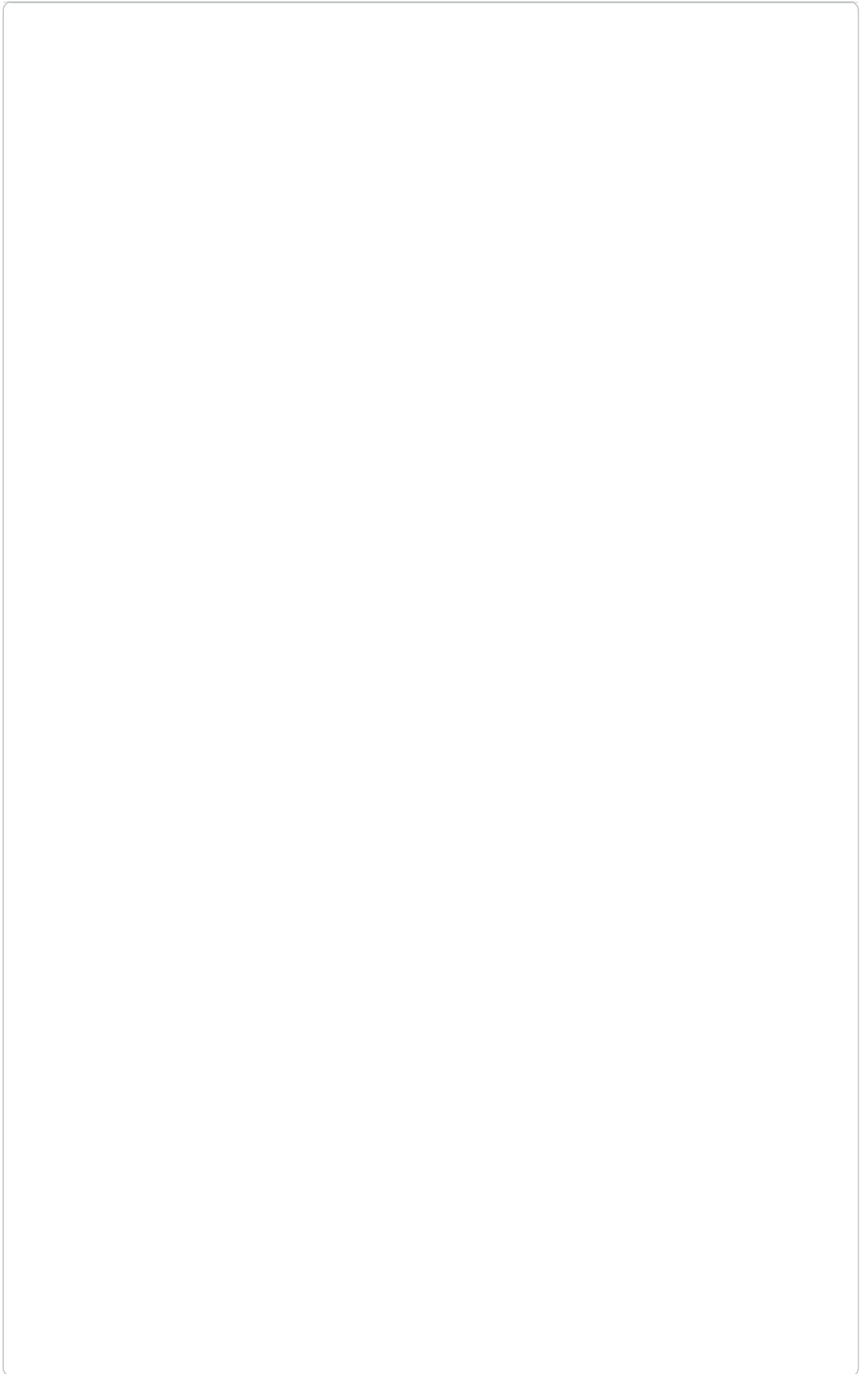
print("\n=====")
print("          FILES SAVED SUCCESSFULLY")
print("=====")

print("1. SIFT_matching_results.csv")
print("2. SIFT_good_matches.jpg")
print("3. SIFT_keypoints_image1.jpg")
print("4. SIFT_keypoints_image2.jpg")

# =====
# STEP 23: FINAL OBSERVATION
# =====

print("\n=====")
print("          CONCLUSION")
print("=====")

print("""
SIFT successfully detected distinctive keypoints
and generated descriptors for the images.
""")
```



Upload Image 1 (Reference Image)

Upload Image 2 (Test Image)

Image 1 - Reference

Image 2 - Test

Image 1 (Reference Image)



Image 2 (Test Image)



===== SIFT FEATURE EXTRACTION =====

Keypoints in Image 1: 1500

Keypoints in Image 2: 1500

Feature extraction time: 367.99 ms

SIFT Keypoints - Image 1

SIFT Keypoints - Image 2

Image 1 (Reference Image)



Image 2 (Test Image)



===== FEATURE MATCHING RESULTS =====

Total candidate matches: 1500

Good matches: 81

Good match percentage: 5.4 %

Matching time: 70.63 ms

SIFT Feature Matching - Good Ma

